

DOCUMENTAÇÃO TÉCNICA DO CÓDIGO

A3 – Estrutura de Dados e Análise de Algoritmos

Sistema de Gerenciamento de Conexões entre Creches utilizando Grafos

1. VISÃO GERAL DO PROJETO

O objetivo do sistema é representar um conjunto de creches e suas conexões através de um grafo ponderado.

Cada creche é representada por um vértice do grafo.

Cada conexão entre duas creches é representada por uma aresta contendo uma distância em quilômetros.

O sistema realiza:

- Leitura dos dados através de arquivo texto;
 - Armazenamento das conexões;
 - Consulta de conexões;
 - Consulta de distâncias;
 - Inclusão de novas conexões.
-

2. ESTRUTURA GERAL DO PROJETO

O projeto foi dividido em quatro classes principais:

```
Main.java
|
├── Grafo.java
|
├── ListaEncadeada.java
|
└── No.java
```

Cada classe possui uma responsabilidade específica.

3. CLASSE No.java

Objetivo

Representar um elemento da lista encadeada.

Cada nó armazena uma conexão entre duas creches.

Estrutura

```
String creche;  
  
double distancia;  
  
No proximo;
```

Atributos

creche

Armazena o nome da creche conectada.

Exemplo:

```
"JoaoGama"
```

distancia

Armazena a distância da conexão.

Exemplo:

```
13.6
```

proximo

Guarda a referência para o próximo nó da lista.

Construtor

```
public No(  
    String creche,
```

```
        double distancia  
    )
```

Responsável por criar um novo nó contendo:

- Nome da creche;
- Distância;
- Referência nula para o próximo elemento.

4. CLASSE ListaEncadeada.java

Objetivo

Gerenciar as conexões de cada creche.

Cada posição do vetor de listas contém uma lista encadeada própria.

Atributo Principal

```
private No inicio;
```

Representa o primeiro elemento da lista.

Método inserir()

```
public void inserir(  
    String creche,  
    double distancia  
)
```

Função

Adicionar uma nova conexão.

Processo

1. Criar um novo nó.
2. Verificar se a lista está vazia.
3. Caso esteja vazia:
4. inserir como primeiro elemento.

5. Caso contrário:

6. percorrer a lista.

7. inserir no final.

Método quantidade()

```
public int quantidade()
```

Função

Contar quantos elementos existem na lista.

Retorno

Número de conexões da creche.

Exemplo:

```
4
```

Método buscarDistancia()

```
public Double buscarDistancia(  
    String destino  
)
```

Função

Procurar a distância para uma creche específica.

Retorno

Se encontrar:

```
13.6
```

Se não encontrar:

```
null
```

Método listarOrdenado()

```
public void listarOrdenado()
```

Função

Ordenar os elementos da lista pela distância.

Algoritmo Utilizado

Selection Sort.

Motivo

Por ser simples e adequado ao tamanho reduzido da lista.

5. CLASSE Grafo.java

Objetivo

Controlar toda a estrutura do grafo.

Esta é a classe principal da lógica do sistema.

Estruturas Utilizadas

Vetor de Creches

```
private String[] creches;
```

Armazena os nomes das creches.

Exemplo:

```
creches[0] = "JoaoGama";  
  
creches[1] = "PedroFernandes";
```

Matriz de Adjacência

```
private int[][] matrizAdj;
```

Representa a existência das conexões.

Exemplo

	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

Onde:

- 0 = sem conexão
- 1 = com conexão

Vetor de Listas

```
private ListaEncadeada[] listas;
```

Cada posição contém as conexões daquela creche.

Exemplo:

```
listas[0]
```

Contém todas as conexões da creche armazenada em:

```
creches[0]
```

Método obterIndice()

```
private int obterIndice(  
    String nome  
)
```

Função

Encontrar o índice de uma creche.

Caso a creche não exista:

- adiciona ao vetor;
- retorna o novo índice.

Método buscarIndice()

```
private int buscarIndice(  
    String nome  
)
```

Função

Localizar uma creche já cadastrada.

Retorno

Se existir:

```
0, 1, 2...
```

Se não existir:

```
-1
```

Método carregarArquivo()

```
public void carregarArquivo(  
    String arquivo  
)
```

Função

Ler o arquivo TXT.

Exemplo

```
JoanaTimoteo;AmaroCavalcante;2.9
```

Processo

1. Ler linha.
2. Separar usando ";"
3. Obter origem.
4. Obter destino.
5. Obter distância.
6. Criar conexão.

Método adicionarConexao()

```
public void adicionarConexao(  
    String origem,  
    String destino,  
    double distancia  
)
```

Função

Adicionar uma nova ligação.

Atualizações realizadas

Matriz

```
matrizAdj[i][j] = 1;  
  
matrizAdj[j][i] = 1;
```


Lista Encadeada

```
listas[i].inserir(...)  
  
listas[j].inserir(...)
```

Como o grafo é não direcionado, ambas as direções são atualizadas.

Método mostrarQuantidadeConexoes()

Função

Exibir o número de conexões de cada creche.

Exemplo:

```
JoaoGama -> 4 conexões  
  
PedroFernandes -> 3 conexões
```

Método listarConexoes()

Função

Exibir as conexões de uma creche em ordem crescente de distância.

Exemplo:

```
PorfiroGabrielDosAnjos - 7.6 km  
  
PedroFernandes - 11.4 km  
  
AnaReinaldo - 13.6 km
```

Método distanciaEntre()

Função

Informar a distância entre duas creches.

Exemplo:

Distância entre PedroFernandes e HerculanoDeMelo:

14.5 km

Método mostrarMatriz()

Função

Exibir a matriz de adjacência.

Exemplo

```
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

6. CLASSE Main.java

Objetivo

Executar e testar o sistema.

Fluxo Principal

1. Criar o Grafo

```
Grafo grafo =
    new Grafo();
```

2. Ler Arquivo

```
grafo.carregarArquivo(  
    "dados.txt"  
);
```

3. Mostrar Conexões

```
grafo.mostrarQuantidadeConexoes();
```

4. Listar Conexões

```
grafo.listarConexoes(  
    "JoaoGama"  
);
```

5. Consultar Distância

```
grafo.distanciaEntre(  
    "PedroFernandes",  
    "HerculanoDeMelo"  
);
```

6. Adicionar Nova Conexão

```
grafo.adicionarConexao(  
    "JoseVicente",  
    "PedroFernandes",  
    8.0  
);
```

7. CONCLUSÃO

O sistema foi desenvolvido utilizando os conceitos de grafos, matriz de adjacência e listas simplesmente encadeadas.

A implementação atende integralmente aos requisitos propostos no trabalho, permitindo o gerenciamento das conexões entre creches e a realização de consultas importantes para o planejamento da distribuição de merenda escolar.

Além disso, o projeto demonstra a aplicação prática dos conteúdos estudados na disciplina de Estrutura de Dados e Análise de Algoritmos, reforçando conceitos fundamentais de organização e manipulação de dados.